



Universidad  
Nacional  
de Loja

**UNIVERSIDAD NACIONAL DE LOJA**

**FACULTAD DE ENERGÍA, LAS INDUSTRIAS Y LOS RECURSOS NATURALES NO**

**RENOVABLES**

**CARRERA DE COMPUTACIÓN**

**“TRABAJO AUTÓNOMO DE TEORÍA DE ÁRBOLES Y SUS APLICACIONES”**

**Estudiantes:**

Andy Ordoñez

Miguel Rojas

David Luna

Karen Lalangui

**Docente:** Ing. Mario Cueva

15 de julio de 2026

## TEMA 1: TERMINOLOGÍA BÁSICA

### Introducción y Aplicaciones

Los árboles son un tipo especial de grafo, muy usados en ciencias de la computación. Casi todos los sistemas operativos guardan sus archivos en una estructura de árbol. El libro menciona las siguientes aplicaciones principales:

- ✓ Organización de información, para poder operar sobre ella eficientemente.
- ✓ Algoritmos de búsqueda eficientes para localizar elementos en una lista.
- ✓ Códigos eficientes para almacenar y transmitir datos (p. ej., código de prefijos, Huffman).
- ✓ Modelación de procedimientos que siguen una secuencia de decisiones.

### Definición de un Árbol

Un árbol es cualquier grafo no dirigido, conexo y sin circuitos (sin ciclos). Para que un grafo sea considerado un árbol debe presentar las siguientes características:

- ✓ No dirigido: sus lados no tienen dirección.
- ✓ Conexo: existe un camino entre cada par de vértices.
- ✓ Sin circuitos: no hay forma de salir de un vértice y volver a él sin repetir un lado.

Propiedades importantes de un árbol:

- ✓ Existe un único paseo entre dos vértices cualesquiera.
- ✓ El número de vértices es mayor que el número de lados.
- ✓ Un árbol con dos o más vértices tiene al menos una hoja.

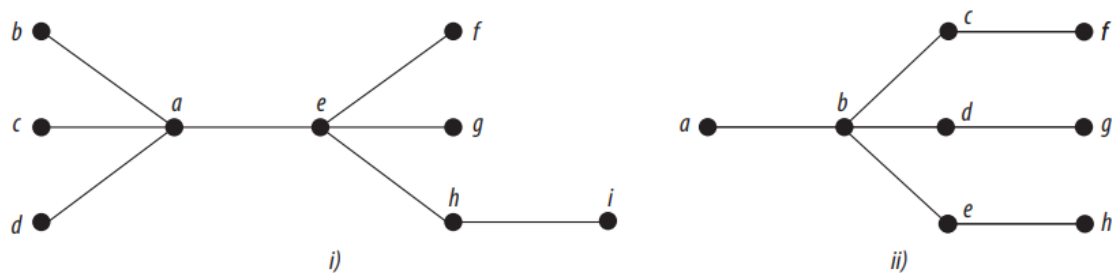


Figura 1

*Ejemplo de un grafo dirigido*

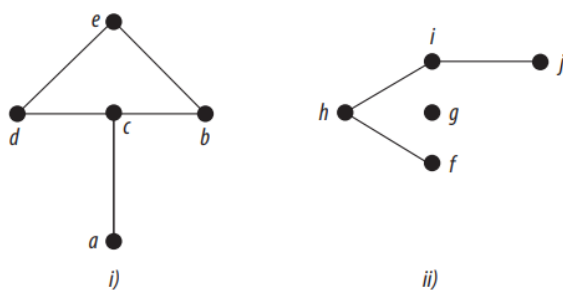


Figura 2

*Ejemplo de un grafo no dirigido*

El grafo ii no es árbol, ya que es desconexo, pues contiene un vértice aislado (vértice g).

Bosque: una colección de árboles disjuntos (que no se tocan entre sí).

### Componentes Principales de un Árbol

- ✓ Nodo (vértice): elemento que almacena información.
- ✓ Arista: conexión entre dos nodos.
- ✓ Nodo hoja: nodo que no tiene hijos (terminal).
- ✓ Nodo interno o rama: nodo que posee uno o más hijos.
- ✓ Raíz: nodo principal desde donde inicia el árbol (en árboles enraizados).

### Tipos de Árboles

*Tabla 1*  
*Clasificación de los tipos de árboles y sus aplicaciones*

| <b>Tipo</b>     | <b>Definición</b>                                                                                                                                  | <b>Aplicaciones</b>                                                                                                               |
|-----------------|----------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| Árbol dirigido  | Grafo dirigido que, al ignorar la dirección de las aristas, sigue siendo un árbol.                                                                 | Representación de flujos de información, dependencias y procesos.                                                                 |
| Árbol enraizado | Árbol dirigido con un único nodo llamado raíz, desde el cual parten todos los demás nodos. Cada nodo (excepto la raíz) tiene exactamente un padre. | Sistemas de archivos, árboles genealógicos, organización empresarial, DOM de HTML.                                                |
| Árbol ordenado  | Árbol enraizado en el que existe un orden específico entre los hijos de cada nodo.                                                                 | Expresiones matemáticas, compiladores, análisis sintáctico.                                                                       |
| Árbol m-ario    | Árbol ordenado donde cada nodo interno puede tener como máximo $m$ hijos. Si todos tienen exactamente $m$ hijos, se denomina m-ario regular.       | Representación de estructuras jerárquicas con múltiples opciones, sistemas de archivos y bases de datos.                          |
| Árbol binario   | Caso particular del árbol m-ario donde $m = 2$ . Cada nodo tiene como máximo dos hijos: izquierdo y derecho.                                       | Árboles de búsqueda binaria (BST), árboles AVL, árboles de expresión, algoritmos de Huffman y recorridos de estructuras de datos. |

## TEMA 2: ÁRBOLES ENRAIZADOS

En el ámbito de las matemáticas discretas, el árbol enraizado constituye una estructura de grafo dirigida de gran importancia para la organización lógica de datos no lineales. A diferencia de los grafos generales, cuya navegación puede carecer de un sentido unívoco, el árbol enraizado establece una jerarquía estricta de vértices a partir de un nodo primigenio, o raíz.

Esta arquitectura, analizada sistemáticamente en las secciones 7.2 y 7.3 de la obra de Villalpando Becerra, permite transformar la interconectividad arbitraria propia de los grafos en un sistema de navegación dirigida y estructurada, indispensable para la gestión eficiente de la información en la ingeniería de software moderna.

La transición de un grafo simple a un árbol enraizado resulta fundamental desde un punto de vista estratégico, pues permite segmentar la complejidad mediante la relación de ascendencia y descendencia entre nodos. Mientras que en los grafos generales (capítulo 6) las trayectorias pueden ser cíclicas, el árbol enraizado impone un orden de “padres e hijos” que garantiza rutas únicas desde la raíz hasta cualquier nodo terminal. Esta propiedad es la que faculta a los sistemas de información para realizar búsquedas deterministas y recorridos jerárquicos sin ambigüedad.

A partir de la terminología técnica establecida, pueden definirse los siguientes elementos constitutivos de esta estructura:

- ✓ **Vértice:** unidad fundamental del árbol; representa un nodo de información o un punto de ramificación dentro de la red.
- ✓ **Raíz:** vértice distintivo y único que actúa como origen de la jerarquía; es el único nodo que carece de predecesor o padre.
- ✓ **Hoja:** vértice terminal que no posee descendientes; representa el punto final de una rama o una unidad de dato atómica.
- ✓ **Subárbol:** estructura recursiva compuesta por un vértice y todos sus descendientes, la cual preserva por sí misma las propiedades de un árbol enraizado.
- ✓ **Árbol m-ario:** tipología de árbol enraizado en la que cada vértice interno posee, como máximo,  $m$  hijos; el árbol binario ( $m = 2$ ) es su exponente más relevante en la computación.

Esta configuración jerárquica constituye el requisito indispensable para cuantificar la eficiencia operativa de un árbol, permitiendo medir con rigor matemático los recorridos que se analizan en la siguiente sección.

### **Evaluación de la Eficiencia: Longitud de Paseo en Árboles**

La evaluación del desempeño de un sistema basado en árboles depende, en buena medida, de la longitud de paseo en árboles enraizados (sección 7.4). Esta métrica cuantifica la suma de las distancias, medidas en número de aristas, desde la raíz hasta cada uno de los vértices del árbol. En la práctica de la ingeniería, la distancia desde la raíz es la variable determinante de la complejidad operativa, pues cada nivel de profundidad adicional representa una operación de búsqueda o un ciclo de reloj extra en el acceso a los datos.

La relación entre la altura de un árbol y su longitud de paseo define el límite de eficiencia de un algoritmo. En un árbol balanceado, como las estructuras AVL analizadas en la sección 7.1.1, se garantiza una altura logarítmica de  $O(\log n)$ , lo que minimiza la longitud de paseo y asegura tiempos de acceso óptimos. Por el contrario, un árbol no balanceado puede degenerar en una estructura lineal con una altura de  $O(n)$ , lo que incrementa drásticamente la longitud de paseo y degrada el rendimiento del sistema hasta asemejarlo a una búsqueda secuencial ineficiente.

En síntesis, el cálculo de la longitud de paseo constituye el indicador primario de la eficiencia estructural de un árbol enraizado. Esta magnitud se obtiene mediante la suma ponderada de los niveles de todos los vértices, y establece una relación directa entre la topología del árbol y el costo computacional requerido para la recuperación de información en sistemas jerárquicos.

Minimizar la longitud de paseo no es una mera abstracción teórica, sino el objetivo central del diseño de estructuras de datos orientadas a la máxima velocidad de respuesta. Esta optimización encuentra su aplicación más directa en la ingeniería de comunicaciones, a través del desarrollo de códigos binarios que eliminan la redundancia y la latencia.

### **Ejemplo Práctico**

Considérese un árbol enraizado con una raíz  $R$  y cinco vértices adicionales, distribuidos en tres niveles:

- ✓ Nivel 0: la raíz R (distancia 0).
- ✓ Nivel 1: los vértices A y B, hijos directos de R (distancia 1 cada uno).
- ✓ Nivel 2: los vértices C y D, hijos de A, y el vértice E, hijo de B (distancia 2 cada uno).

La longitud de paseo total se obtiene sumando la distancia de cada vértice a la raíz:

$$L = 0 (R) + 1 (A) + 1 (B) + 2 (C) + 2 (D) + 2 (E) = 8$$

Si, en cambio, los cinco vértices A, B, C, D y E se organizaran en una única cadena lineal descendente desde la raíz (es decir, un árbol degenerado con altura  $O(n)$ ), la longitud de paseo sería  $L = 1 + 2 + 3 + 4 + 5 = 15$ . La comparación evidencia, con un caso concreto, cómo el balanceo del árbol reduce casi a la mitad el costo total de acceso a los datos, confirmando la relación entre altura y longitud de paseo descrita anteriormente.

### **Aplicaciones de Codificación: el Código de Prefijos**

El código de prefijos, expuesto en la sección 7.5, representa una aplicación destacada de los árboles binarios en el ámbito de la transmisión de datos. Su propósito es asignar secuencias binarias a símbolos de manera que la decodificación sea instantánea y libre de ambigüedad. En este esquema, el árbol binario funciona como un mapa de decisiones en el que cada hoja representa un símbolo específico, y el camino desde la raíz hasta esa hoja determina su representación binaria única.

La ventaja diferencial de un código de prefijos radica en su propiedad de ser “libre de prefijo”: ningún código asignado a un símbolo es, a la vez, el inicio del código de otro símbolo. Esta característica, derivada de una estructura en la que los datos residen únicamente en las hojas, garantiza que un sistema de comunicación pueda identificar un mensaje tan pronto como se reciba el último bit, sin necesidad de marcadores de fin de bloque. De este modo se optimiza el ancho de banda y se elimina la necesidad de procesos de “look-ahead” o de reanálisis de la cadena de bits.

La siguiente tabla contrasta las características de la codificación convencional frente a las de la codificación de prefijos:

*Tabla 2*

*Comparación entre el código estándar y el código de prefijos*

| <b>Criterio de comparación</b>  | <b>Código estándar (longitud fija)</b>                                     | <b>Código de prefijos (estructura de árbol)</b>                                |
|---------------------------------|----------------------------------------------------------------------------|--------------------------------------------------------------------------------|
| Ambigüedad                      | Requiere delimitadores externos para separar símbolos en ráfagas de datos. | Nula; la propiedad de prefijo garantiza una interpretación única del flujo.    |
| Eficiencia de decodificación    | Depende de la sincronización de bloques; mayor latencia de procesamiento.  | Máxima; decodificación en tiempo real al alcanzar un nodo hoja.                |
| Impacto de la longitud de paseo | Uniforme; no aprovecha la frecuencia de los datos.                         | Directo; permite minimizar la longitud de paseo para los símbolos más comunes. |

### **Ejemplo Práctico**

Supóngase que un sistema debe transmitir un mensaje compuesto únicamente por los símbolos A, B, C y D, con las siguientes frecuencias de aparición: A = 50 %, B = 30 %, C = 15 % y D = 5 %. Aplicando el principio de la codificación de prefijos, los símbolos más frecuentes se colocan en las hojas más cercanas a la raíz, y los menos frecuentes en las hojas más profundas, con el fin de minimizar la longitud de paseo ponderada:

*Tabla 3*

*Asignación de códigos de prefijo según la frecuencia de los símbolos*

| <b>Símbolo</b> | <b>Frecuencia</b> | <b>Nivel en el árbol</b> | <b>Código de prefijo</b> |
|----------------|-------------------|--------------------------|--------------------------|
| A              | 50 %              | 1                        | 0                        |
| B              | 30 %              | 2                        | 10                       |
| C              | 15 %              | 3                        | 110                      |



| Símbolo | Frecuencia | Nivel en el árbol | Código de prefijo |
|---------|------------|-------------------|-------------------|
| D       | 5 %        | 3                 | 111               |

Con esta asignación, la cadena “AABCA” se codifica, símbolo a símbolo, como  $0 + 0 + 10 + 110 + 0$ , es decir, la secuencia 00101100, y puede decodificarse de manera única leyendo bit a bit de izquierda a derecha, sin necesidad de separadores: en cuanto se completa un código válido (una hoja), se reconoce el símbolo y se reinicia la lectura desde la raíz. Obsérvese además que ningún código es prefijo de otro (0, 10, 110 y 111), lo que garantiza la ausencia de ambigüedad. Al asignar el código más corto (un solo bit) al símbolo más frecuente (A), se reduce el número total de bits transmitidos en comparación con un esquema de longitud fija de dos bits por símbolo, que requeriría 10 bits para codificar los mismos cinco símbolos.

A través de estas implementaciones, la teoría de grafos se convierte en una herramienta de ingeniería de software fundamental, que dota a los sistemas de comunicación de la robustez necesaria para la transmisión masiva de información.

## Conclusiones

La integración de la teoría de árboles enraizados, la métrica de longitud de paseo y la implementación de códigos de prefijos demuestra que la matemática discreta es el lenguaje fundamental de la optimización algorítmica. El rigor en la definición de jerarquías y en la cuantificación de las distancias estructurales permite a los ingenieros diseñar sistemas que no solo son funcionales, sino matemáticamente eficientes incluso bajo condiciones de alta demanda de recursos.

De este análisis se desprenden tres lecciones fundamentales para el diseño de sistemas de información:

- ✓ La estructura determina la velocidad: mantener un árbol balanceado (AVL) es imperativo para preservar una complejidad de  $O(\log n)$  y evitar la degradación del sistema hacia estructuras lineales de costo  $O(n)$ .
- ✓ La optimización de rutas es esencial: la reducción sistemática de la longitud de paseo es el camino matemático más directo para minimizar el costo operativo en los procesos de búsqueda, inserción y recuperación de datos.
- ✓ La arquitectura libre de prefijos garantiza integridad: el uso de las hojas de un árbol binario para la codificación elimina la ambigüedad en la transmisión, asegurando que la eficiencia del software esté respaldada por la topología del grafo.

En definitiva, el estudio estructural de los árboles, según el paradigma de Villalpando Becerra, confirma que la organización lógica es la piedra angular de la ingeniería de software. El dominio de estas estructuras permite transformar la abstracción matemática en sistemas de computación de alto rendimiento.

### **TEMA 3: ÁRBOLES DE BÚSQUEDA BINARIA (ABB)**

#### **Definición Estructural y Rigor Teórico**

En el marco de las estructuras de datos no lineales, un Árbol de Búsqueda Binaria (ABB) constituye una solución dinámica diseñada para optimizar los procedimientos de localización y ordenamiento de información. De acuerdo con Villalpando Becerra y García Sandoval (2014), un árbol binario común se transforma en un árbol de búsqueda binaria si y solo si cumple rigurosamente con una condición de ordenamiento jerárquico en cada uno de sus nodos.

Formalmente, para cualquier nodo raíz o nodo rama que almacene una clave o etiqueta determinada ( $K_i$ ), se deben verificar las siguientes dos propiedades:

- ✓ Todas las claves localizadas en el subárbol izquierdo de dicho nodo deben ser estrictamente menores en valor que la clave del nodo padre ( $K_i$ ).
- ✓ Todas las claves ordenadas en el subárbol derecho deben ser estrictamente mayores en valor que la clave del nodo padre ( $K_i$ ).

Esta naturaleza recursiva garantiza que cada subárbol sea, a su vez, un árbol de búsqueda binaria. Para comprender la importancia de este rigor conceptual, Villalpando Becerra y García Sandoval (2014) proponen un criterio de validación clásico: si un árbol posee una raíz con el valor numérico 10, y en su subárbol derecho se posiciona la clave 6, dicha estructura deja de ser un árbol de búsqueda binaria. Esto ocurre porque, a pesar de estar físicamente a la derecha, el valor absoluto 6 es menor que la raíz principal 10, lo que invalida la topología de la estructura y exige su posicionamiento mandatorio en el subárbol izquierdo.

### **Lógica Interna de las Operaciones Fundamentales**

Para garantizar la integridad y eficiencia del ABB, los algoritmos de manipulación de datos operan bajo una lógica determinista que preserva las propiedades de orden del árbol en todo momento. A continuación, se analiza el comportamiento de sus tres operaciones principales:

- ✓ Búsqueda de un nodo: el procedimiento algorítmico inicia de forma sistemática en el nodo raíz del árbol, comparando el objeto o clave objetivo ( $x$ ) con la etiqueta de la raíz ( $K_i$ ). Si  $x$  es igual a  $K_i$ , la búsqueda concluye con éxito. En caso de no haber coincidencia, la propiedad del árbol guía el flujo del programa: si  $x < K_i$ , se descarta el lado derecho y se busca recursivamente en el hijo izquierdo; si  $x > K_i$ , se realiza la misma acción en el hijo derecho. Este descenso continúa hasta hallar una concordancia o hasta alcanzar un nodo hoja vacío (NULL), lo que demuestra formalmente la inexistencia del elemento en la estructura.

- ✓ Inserción de un nodo: la inserción de nuevos elementos en un ABB se caracteriza por ser acumulativa y realizarse exclusivamente en los extremos del árbol (nodos hoja). El algoritmo ejecuta primero una búsqueda para comprobar si la clave ya reside en el árbol; si el elemento no se encuentra, la inserción se efectúa exactamente en la posición vacía donde el proceso de búsqueda finalizó. Por ejemplo, al insertar el nodo 6 en un árbol con raíz 24 e hijo izquierdo 8, el algoritmo evalúa las desigualdades ( $6 < 24$  y  $6 < 8$ ), localizando el puntero vacío a la izquierda de 8 para enlazar allí el nuevo nodo hijo.
- ✓ Eliminación de un nodo: esta operación presenta la mayor complejidad lógica, puesto que la supresión de un dato no debe corromper la continuidad y jerarquía del árbol. Si el nodo a eliminar cuenta con menos de dos hijos (es decir, es un nodo hoja o posee un único descendiente), la operación se reduce a desconectar el elemento de la estructura y reajustar los punteros de sus antecesores inmediatos para mantener la cohesión del camino de búsqueda.

### **Comportamiento, Eficiencia y Conexión Estructural**

El rendimiento computacional de los Árboles de Búsqueda Binaria está directamente supeditado a la altura ( $h$ ) de la estructura. En condiciones ideales de balanceo, donde los subárboles mantienen una distribución simétrica de sus nodos, el tiempo requerido para efectuar búsquedas, inserciones o eliminaciones es de naturaleza logarítmica, representado bajo la notación asintótica como  $O(\log n)$ . Esto ofrece una ventaja sustancial sobre las estructuras lineales tradicionales. Sin embargo, si los datos son insertados en un orden estrictamente secuencial, el árbol experimenta una degeneración estructural que lo transforma en una lista enlazada simple, degradando su eficiencia a un comportamiento lineal  $O(n)$ .

Finalmente, es indispensable señalar la profunda interconexión que posee este tema con el estudio de los recorridos en árboles binarios (Tema 4). Mientras que la lógica del ABB organiza y almacena la información de manera selectiva, son los métodos de recorrido en profundidad los que permiten el acceso sistemático a los datos. Específicamente, si se aplica un recorrido en Inorden (recorriendo el subárbol izquierdo, procesando la raíz y concluyendo en el subárbol derecho) sobre un Árbol de Búsqueda Binaria, los elementos almacenados se recuperarán en un orden secuencial estrictamente ascendente, unificando la teoría de la búsqueda con la de los recorridos jerárquicos.

### **Ejemplo Práctico de Construcción y Operaciones en un ABB**

Para ilustrar la metodología descrita por Villalpando Becerra y García Sandoval (2014), se presenta a continuación el desarrollo paso a paso de la construcción de un Árbol de Búsqueda Binaria y la posterior ejecución de operaciones sobre el mismo.

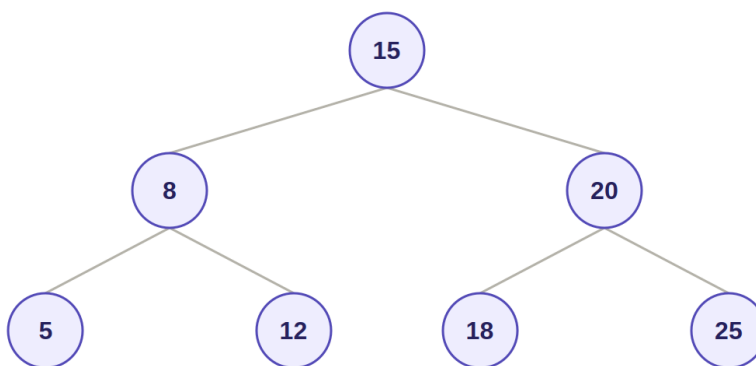
#### ***Construcción del Árbol (Inserciones Sucesivas)***

Se desea construir un ABB utilizando la siguiente secuencia de claves de entrada en el orden exacto en que se presentan: [15, 8, 20, 5, 12, 18, 25].

- ✓ Inserción de 15: al estar el árbol vacío, el primer elemento se convierte automáticamente en el nodo raíz.
- ✓ Inserción de 8: se compara con la raíz ( $8 < 15$ ). Al no existir hijo izquierdo, el 8 se inserta en esa posición.
- ✓ Inserción de 20: se compara con la raíz ( $20 > 15$ ). Se inserta como hijo derecho de la raíz.
- ✓ Inserción de 5: se compara con la raíz ( $5 < 15$ , rama izquierda) y luego con el nodo 8 ( $5 < 8$ ). Se inserta como hijo izquierdo de 8.
- ✓ Inserción de 12: se compara con la raíz ( $12 < 15$ , rama izquierda) y con el nodo 8 ( $12 > 8$ ). Se inserta como hijo derecho de 8.

- ✓ Inserción de 18: se compara con la raíz ( $18 > 15$ , rama derecha) y con el nodo 20 ( $18 < 20$ ). Se inserta como hijo izquierdo de 20.
- ✓ Inserción de 25: se compara con la raíz ( $25 > 15$ , rama derecha) y con el nodo 20 ( $25 > 20$ ). Se inserta como hijo derecho de 20.

El árbol resultante de este proceso estructural se representa de la siguiente manera:



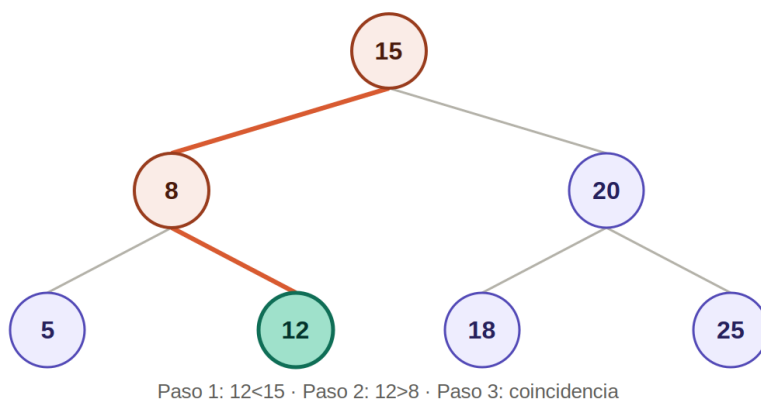
*Figura 3*

*Árbol de búsqueda binaria resultante de las inserciones sucesivas*

### **Ejemplo de Búsqueda**

Si se requiere buscar la clave  $x = 12$  en el árbol construido:

- ✓ Se compara 12 con la raíz 15. Como  $12 < 15$ , el algoritmo se desplaza al subárbol izquierdo.
- ✓ Se compara 12 con el nodo 8. Como  $12 > 8$ , el algoritmo se desplaza al subárbol derecho.
- ✓ Se compara 12 con el nodo 12. Al ser idénticos, la búsqueda finaliza con éxito en solo tres pasos, demostrando la eficiencia logarítmica de la estructura.

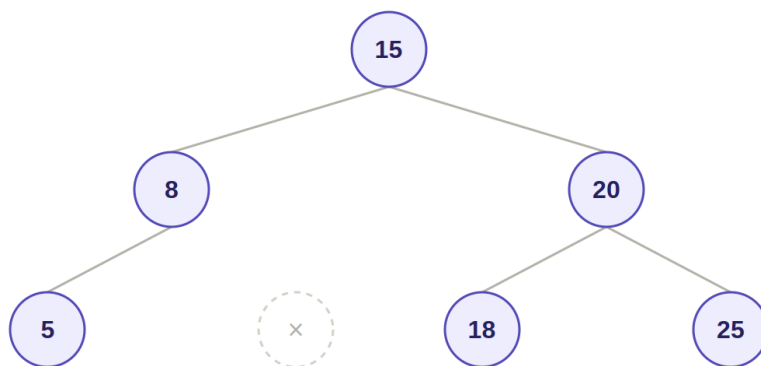


*Figura 4*

*Recorrido de búsqueda de la clave 12*

### ***Ejemplo de Eliminación (Caso Nodo Hoja)***

Si se solicita eliminar la clave 12, dado que el nodo 12 es un nodo hoja (no tiene descendientes), la lógica establecida por Villalpando Becerra y García Sandoval (2014) indica que simplemente se elimina el nodo de la estructura y se limpia el puntero derecho de su padre (el nodo 8). El árbol resultante mantiene perfectamente sus propiedades.



*Figura 5*

*Árbol resultante tras la eliminación del nodo 12*

## TEMA 4: RECORRIDOS EN UN ÁRBOL

Basado en el capítulo 7 (que inicia en la página 241) del libro Matemáticas Discretas: Aplicaciones y Ejercicios de José Francisco Villalpando Becerra, el análisis de los recorridos en profundidad de un árbol binario se estructura mediante una nomenclatura muy específica.

Villalpando define la metodología basándose en el orden exacto en el que el algoritmo visita tres elementos: el nodo raíz (N), el subárbol izquierdo (I) y el subárbol derecho (D).

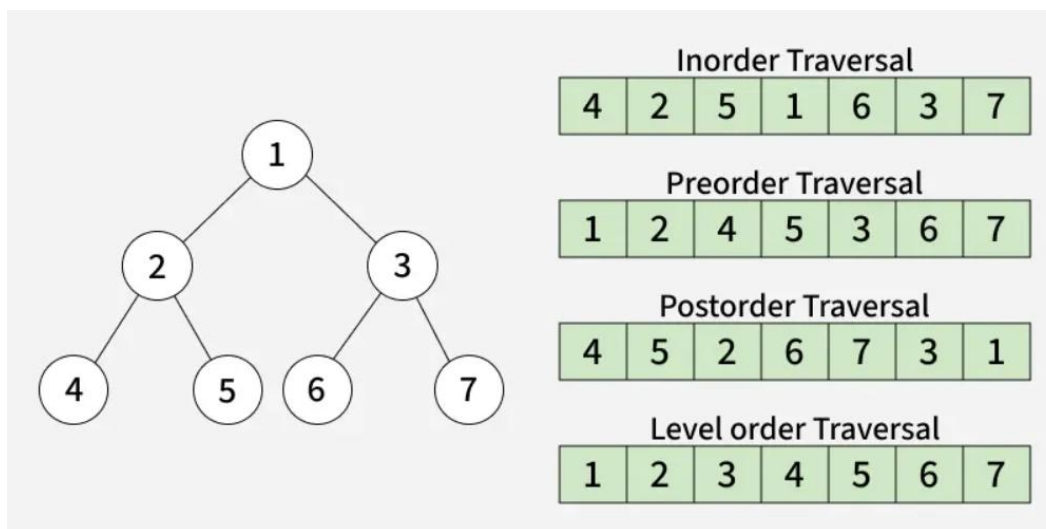
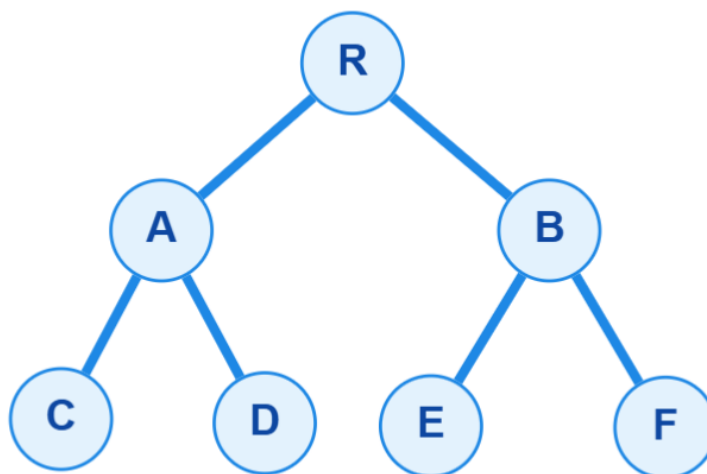


Figura 6

*Nomenclatura de los recorridos en profundidad*

### Resolución Analítica Paso a Paso





*Figura 7*

*Árbol de ejemplo (Figura 19 del texto de referencia)*

Para ilustrar la metodología de Villalpando, se utiliza la estructura del árbol de ejemplo de su texto (conocido en el libro como la Figura 19). Consiste en un árbol con la raíz principal *r*, un subárbol izquierdo encabezado por *a*, y un subárbol derecho encabezado por *b*:

### **Recorrido en Preorden (NID)**

Regla metodológica: nodo, izquierdo, derecho (NID). Se visita primero la raíz, luego se recorre todo el subárbol izquierdo aplicando la misma regla, y finalmente el derecho.

- Paso 1 (N): se visita la raíz principal: *r*.
- Paso 2 (I): se pasa al subárbol izquierdo (raíz *a*). Se aplica NID localmente: se visita el nodo *a*, luego su hijo izquierdo *c*, y después su hijo derecho *d*.
- Paso 3 (D): se pasa al subárbol derecho (raíz *b*). Se aplica NID localmente: se visita el nodo *b*, luego su hijo izquierdo *e*, y después su hijo derecho *f*.

Secuencia final: *r - a - c - d - b - e - f*

### **Recorrido en Enorden o Inorden (IND)**

Regla metodológica: izquierdo, nodo, derecho (IND). Se recorre el subárbol izquierdo hasta su extremo, se visita la raíz, y luego se pasa al derecho.

- ✓ Paso 1 (I): en el subárbol izquierdo de r, se baja hasta el nodo más a la izquierda: c.
- ✓ Paso 2 (N): se sube a visitar su nodo padre/raíz local: a.
- ✓ Paso 3 (D): se visita el hijo derecho de a: d.
- ✓ Paso 4 (N): con el subárbol izquierdo completo, se visita la raíz principal del árbol: r.
- ✓ Paso 5 (I, N, D): se aplica la regla al subárbol derecho. Primero el izquierdo (e), luego su nodo padre (b), y al final el derecho (f).

Secuencia final: c - a - d - r - e - b - f

### ***Recorrido en Postorden (IDN)***

Regla metodológica: izquierdo, derecho, nodo (IDN). Se procesan primero las ramas inferiores (de izquierda a derecha) y la evaluación de la raíz se deja estrictamente para el final.

- ✓ Paso 1 (I): en el subárbol izquierdo se procesan primero las hojas y al final su raíz local: izquierdo (c), derecho (d), nodo (a).
- ✓ Paso 2 (D): en el subárbol derecho se aplica la misma lógica: izquierdo (e), derecho (f), nodo (b).
- ✓ Paso 3 (N): finalmente se visita la raíz principal de todo el árbol: r.

- Secuencia final: c - d - a - e - f - b - r

## **RESOLUCIÓN DE EJERCICIOS**

### **Ejercicio 1: Árbol de Expresión Aritmética**

Los árboles binarios se usan comúnmente para evaluar expresiones matemáticas. Este árbol representa la expresión  $(A + B) * (C - (D / E))$ .

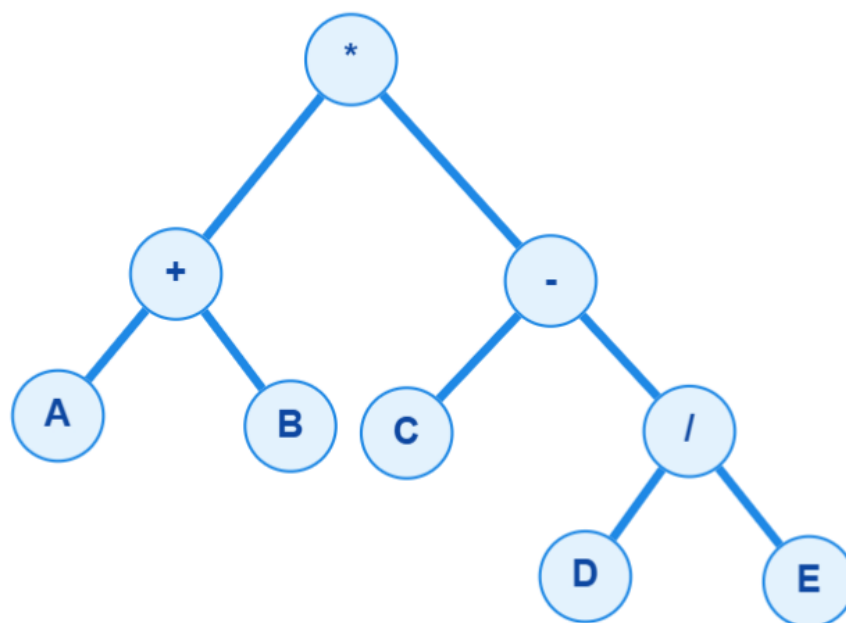


Figura 8

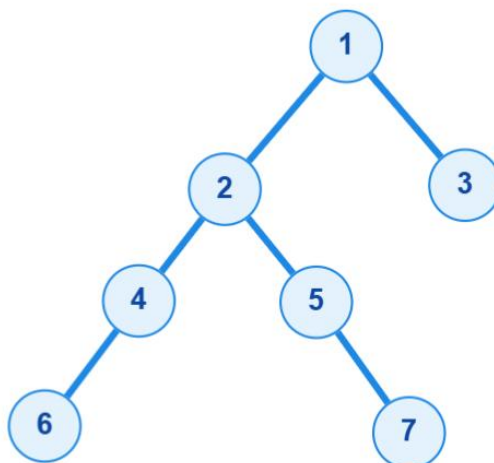
*Árbol de expresión aritmética*

### **Resolución**

- ✓ Preorden (NID): se empieza en la raíz \*, luego todo el lado izquierdo (+ A B), y luego el derecho (- C / D E). Resultado: \* + A B - C / D E.
- ✓ Inorden (IND): extremo izquierdo (A), su padre (+), el derecho (B); luego la raíz principal (\*); luego el lado derecho procesado igual. Resultado: A + B \* C - D / E. El inorden en árboles de expresión genera la notación infija estándar, aunque sin paréntesis.
- ✓ Postorden (IDN): se resuelven las hojas de izquierda a derecha; primero A y B con su nodo +, luego el subárbol derecho (C, D, E, /, -), y al final la raíz \*. Resultado: A B + C D E / - \*.

### **Ejercicio 2: Árbol Desbalanceado a la Izquierda**

Este árbol tiene una tendencia marcada hacia las ramas izquierdas, lo que cambia la forma en que se percibe el recorrido.



*Figura 9*

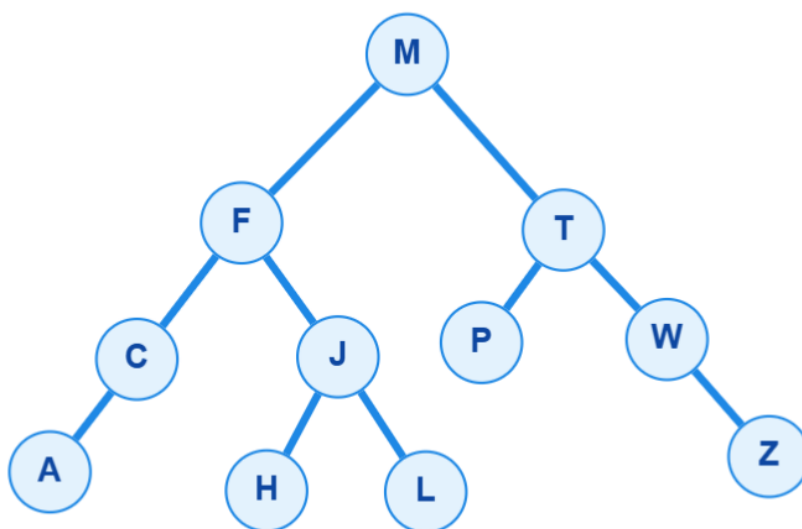
*Árbol desbalanceado a la izquierda*

### **Resolución**

- ✓ Preorden (NID): 1 (nodo), subárbol izquierdo de 1 (2, 4, 6, 5, 7), y finalmente el hijo derecho 3. Resultado: 1 - 2 - 4 - 6 - 5 - 7 - 3.
- ✓ Inorden (IND): se baja hasta el 6, se sube a 4, no hay derecho, se sube a 2, se baja al izquierdo de 5 (vacío), nodo 5, derecho 7, raíz 1, derecho 3. Resultado: 6 - 4 - 2 - 5 - 7 - 1 - 3.
- ✓ Postorden (IDN): se empieza por la hoja 6, se sube a su padre 4; luego se va a la hoja 7, su padre 5, luego el padre 2; luego hoja 3; finalmente raíz 1. Resultado: 6 - 4 - 7 - 5 - 2 - 3 - 1.

### **Ejercicio 3: Árbol Binario de Búsqueda (BST) Alfabetizado**

En un árbol de búsqueda binaria, el hijo izquierdo es menor que el padre, y el derecho es mayor. En este ejercicio se emplean letras.



*Figura 10*

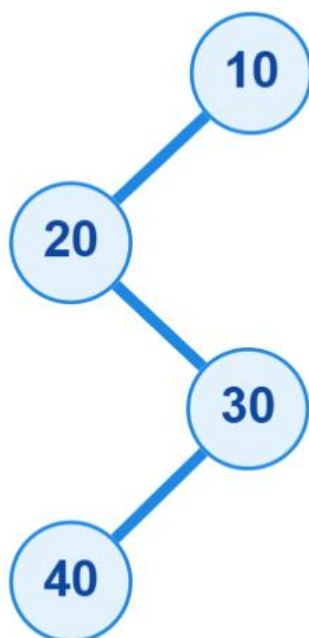
*Árbol binario de búsqueda alfabetizado*

### **Resolución**

- ✓ Preorden (NID): M (raíz), subárbol izquierdo (F, C, A, J, H, L), subárbol derecho (T, P, W, Z). Resultado: M - F - C - A - J - H - L - T - P - W - Z.
- ✓ Inorden (IND): como es un árbol de búsqueda, el recorrido inorden siempre devuelve los elementos ordenados (de la A a la Z). Resultado: A - C - F - H - J - L - M - P - T - W - Z.
- ✓ Postorden (IDN): primero las hojas de la rama izquierda (A, luego C), luego las hojas del subárbol de J (H, L, J), el nodo F, y misma lógica en el lado derecho. Resultado: A - C - H - L - J - F - P - Z - W - T - M.

### **Ejercicio 4: Árbol “Zig-Zag”**

Este árbol alterna entre hijos únicos izquierdos y derechos, generando una estructura larga, pero con pocos nodos.



*Figura 11*

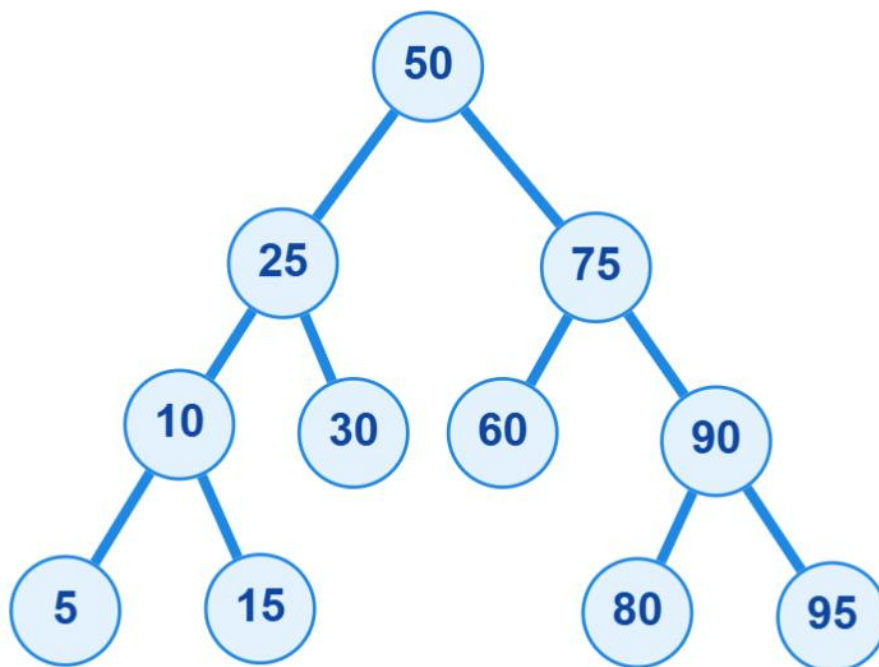
*Árbol en zig-zag*

### ***Resolución***

- ✓ Preorden (NID): se visitan los nodos conforme se desciende, ya que no hay ramas paralelas. Resultado: 10 - 20 - 30 - 40.
- ✓ Inorden (IND): se baja por el 10, luego al 20 (tiene izquierdo vacío, así que se visita el nodo 20), se pasa a su derecho 30 (tiene izquierdo 40, así que se visita 40), se sube a 30 para visitarlo, y finalmente se procesa la raíz 10. Resultado: 20 - 40 - 30 - 10.
- ✓ Postorden (IDN): se baja hasta el último hijo disponible (40), se visita, y se retrocede nodo por nodo procesando cada uno en el camino de regreso hacia la raíz. Resultado: 40 - 30 - 20 - 10. Nota: a pesar de la estructura lineal en zig-zag, debido a la evaluación estricta de los subárboles vacíos de cada nodo, los recorridos inorden y postorden no coinciden.

### Ejercicio 5: Árbol Estrictamente Binario (Lleno hasta el Nivel 2)

Un árbol donde cada nodo tiene exactamente 0 o 2 hijos.



*Figura 12*

*Árbol estrictamente binario*

#### **Resolución**

- ✓ Preorden (NID): raíz 50, luego se procesa toda la mitad izquierda completa, luego la derecha. Resultado: 50 - 25 - 10 - 5 - 15 - 30 - 75 - 60 - 90 - 80 - 95.
- ✓ Inorden (IND): extremo izquierdo (5), padre (10), derecho (15); luego se sube al padre superior (25), derecho (30); raíz principal 50; mitad derecha. Resultado: 5 - 10 - 15 - 25 - 30 - 50 - 60 - 75 - 80 - 90 - 95.
- ✓ Postorden (IDN): se agrupa por hojas inferiores: 5 y 15 con su padre 10, luego el 30, todos bajo el 25; misma lógica en la derecha; raíz al final. Resultado: 5 - 15 - 10 - 30 - 25 - 60 - 80 - 95 - 90 - 75 - 50.

## **Conclusión**

El estudio de los árboles binarios y sus métodos de recorrido evidencia cómo los modelos de la matemática discreta constituyen el pilar fundamental para el diseño de software eficiente, permitiendo transicionar de estructuras lineales a jerárquicas capaces de optimizar las operaciones de búsqueda a una complejidad de  $O(\log n)$ . La aplicación de recorridos sistemáticos como Inorden, Preorden y Postorden demuestra que la forma de procesar la información responde a propósitos específicos de la ingeniería, facilitando desde la recuperación ordenada de datos hasta la optimización de memoria. En última instancia, esta abstracción teórica resulta indispensable para el desarrollo de soluciones reales en computación, siendo la base directa de sistemas de archivos, indexación en motores de bases de datos y compiladores.

## **Reflexión Crítica**

### **Karen Lalangui**

El estudio de los árboles demuestra que las estructuras más organizadas suelen ser también las más eficientes. Comprender los conceptos de raíz, nodos, hojas y ramas permite reconocer cómo muchas de las tecnologías que se utilizan diariamente, como los sistemas de archivos, las bases de datos y los algoritmos de búsqueda, dependen de esta estructura para funcionar de manera rápida y ordenada. Además, conocer los diferentes tipos de árboles evidencia que cada uno responde a necesidades específicas, adaptándose a distintos problemas del mundo de la informática.

### **David Luna**

Al desarrollar esta práctica quedó claro que un árbol es un caso particular de grafo (conexo y sin ciclos) y que precisamente esa restricción es lo que lo vuelve tan útil para organizar información de forma jerárquica y eficiente. El repaso de los fundamentos y tipos de árboles



permitió ordenar conceptos que antes se manejaban de forma confusa, como la diferencia entre árbol libre y enraizado, mientras que el análisis de la longitud de camino, los códigos de Huffman y los árboles de búsqueda binaria dejó ver que decisiones aparentemente simples, como balancear un árbol o decidir “menor a la izquierda, mayor a la derecha”, tienen detrás una justificación matemática orientada a la eficiencia. Sin embargo, lo que más trabajo costó y a la vez más enseñó fue la parte práctica de los recorridos Preorden, Inorden y Postorden, ya que diseñar y resolver ejercicios propios con árboles de cuatro niveles exigió disciplina en el seguimiento recursivo y evidenció errores que antes se cometían por apresurarse, dejando claro que los árboles son, más que una estructura de datos, una forma de organizar el pensamiento para resolver problemas paso a paso.

### **Miguel Rojas**

A nivel personal, el estudio detallado de la teoría de árboles y sus diferentes métodos de recorrido (Preorden, Inorden y Postorden) ha transformado mi perspectiva sobre la organización y manipulación de información no lineal. Como estudiante de computación, este trabajo demostró que la eficiencia de un programa no depende únicamente del código que se escribe, sino del rigor de la estructura matemática en la que se sostiene. Comprender cómo la altura de un Árbol de Búsqueda Binaria (ABB) influye directamente en el costo de búsqueda de un dato, garantizando un tiempo óptimo de  $O(\log n)$  en estructuras balanceadas frente a la degradación de  $O(n)$  cuando el árbol se degenera, aporta un criterio técnico crucial para el diseño de bases de datos y sistemas de archivos eficientes. Asimismo, analizar aplicaciones prácticas como la codificación de prefijos permite ver con claridad cómo la abstracción de la matemática discreta hace posible optimizar el ancho de banda y la transmisión de datos en el software moderno.

### **Andy Ordoñez**

El estudio de los recorridos en árboles binarios permitió comprender que la estructuración de datos va mucho más allá de una simple organización teórica, consolidándose como una pieza fundamental para la eficiencia algorítmica en la ingeniería de software. Al analizar y aplicar detalladamente las metodologías de recorrido en profundidad (Preorden, Inorden y Postorden), resulta evidente cómo el orden exacto de visita a los nodos (raíz, subárbol izquierdo y subárbol derecho) impacta directamente en la forma en que el computador procesa, busca y recupera la información. Más allá de resolver jerarquías abstractas, dominar estas técnicas fomenta un pensamiento lógico riguroso que es vital para el diseño de bases de datos, motores de búsqueda y compiladores, demostrando que la correcta navegación estructural de los datos es la clave para reducir tiempos de procesamiento y construir sistemas computacionales verdaderamente optimizados y escalables.

### **Portafolio**

Karen Lalangui:

<https://github.com/valentinalalangui05-source/Portafolio-Matematicas-discretas>

David Luna:

<https://github.com/Davidluna28/Portafolio-de-Matem-ticas-Discretas->

Miguel Rojas:

<https://github.com/MiguelRojas2006/portafolioMD>

Andy Ordoñez:

[https://drive.google.com/drive/folders/1ZnNuOE8Wyk8PjAuAYPoVGxpRxXbpEbo6?d  
mr=1&ec=wgc-drive-%5Bmodule%5D-goto](https://drive.google.com/drive/folders/1ZnNuOE8Wyk8PjAuAYPoVGxpRxXbpEbo6?dmr=1&ec=wgc-drive-%5Bmodule%5D-goto)

**Referencias**

Rosen, K. H. (2012). *Discrete mathematics and its applications* (7.<sup>a</sup> ed.). McGraw-Hill.

Villalpando Becerra, J. F., & García Sandoval, Á. B. (2014). *Matemática discreta para computación* (1.<sup>a</sup> ed.). Grupo Editorial Patria.